

# Mikroprocesorová technika

Prednáška č.1: História a vznik mikrokontrolérov

**prof. Ing. Ivan Virgala, PhD.**

Technická univerzita v Košiciach  
Strojnícka fakulta

Katedra priemyselnej automatizácie a mechatroniky  
**ARM-lab** (Applied Robotics and Mechatronics Laboratory)



| Týždeň | Prednáška                         | Cvičenie                            |
|--------|-----------------------------------|-------------------------------------|
| 1      | História a vznik mikrokontrolérov | Číselné sústavy, VS Code, UNO R3    |
| 2      | ATmega328P a digitálne I/O        | LED, tlačidlo, DDRx, PORTx, PINx    |
| 3      | Architektúra a pamäte             | bitové masky, stavový automat       |
| 4      | Prerušenia                        | tlačidlo, inkrementálny enkodér     |
| 5      | Časovače a čítače                 | časová základňa, meranie frekvencie |
| 6      | PWM a DC motor                    | otvorené riadenie rýchlosti         |
| 7      | PID/PSD regulácia DC motora       | uzavretá slučka s enkodérom         |
| 8      | Servopohony                       | RC servo alebo DC servopohon        |
| 9      | Krokové motory                    | STEP/DIR, rampový profil            |
| 10     | ADC a snímače                     | meranie, filtrovanie, kalibrácia    |
| 11     | USART                             | diagnostika a príkazy               |
| 12     | SPI a TWI/I <sup>2</sup> C        | nízkoúrovňový ovládač periférie     |
| 13     | Integrácia a spoľahlivosť         | záverečná integrovaná úloha         |



- Vysvetliť rozdiel medzi mikroprocesorom, mikrokontrolérom a vnoreným (embedded) systémom,
- Zaradiť hlavné historické míľniky: elektrónky, tranzistor, IC, Intel 4004, 8008, 8080, TMS1000, 8086, 8051, AVR,
- Pochopiť, prečo je pri mikrokontroléri rozhodujúca integrácia periférií,
- Rozumieť, prečo budeme pracovať priamo s registrami ATmega328P,
- Vedieť, prečo Arduino UNO R3 používame ako vývojovú dosku, nie ako Arduino framework.



## Používame

- Arduino UNO R3 ako spoľahlivú dosku,
- ATmega328P ako cieľový mikrokontrolér,
- VS Code, PlatformIO, AVR-GCC, AVR-LibC,
- jazyk C a vlastný `main()`.

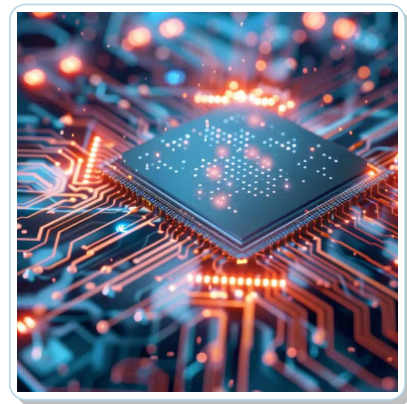
## Nepoužívame

- `setup()`, `loop()`, `.ino`,
- `digitalWrite()`, `analogRead()`, `delay()`,
- hotové knižnice,
- periférie ako čiernu skrinku.

## 💡 Definícia

Vnorený systém je počítačový systém zabudovaný do väčšieho technického zariadenia, ktorý vykonáva konkrétnu meráciu, riadiacu, komunikačnú alebo bezpečnostnú funkciu.

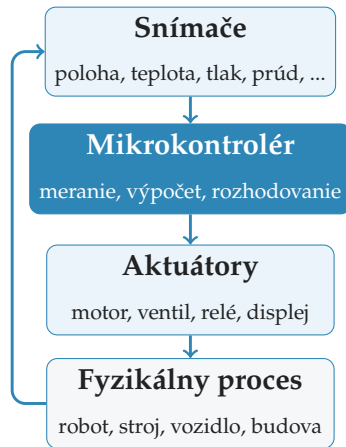
- nie je to univerzálny osobný počítač
- pracuje s fyzikálnymi veličinami
- často pracuje v reálnom čase
- musí reagovať spoľahlivo aj pri poruche



## Základná myšlienka

Mikrokontrolér prepája algoritmus so skutočným fyzikálnym svetom. Meria, rozhoduje a prostredníctvom aktuátorov ovplyvňuje správanie technického systému.

- spracúva údaje zo snímačov,
- vykonáva riadiaci algoritmus,
- ovláda pohony, ventily, svetlá alebo displeje,
- komunikuje s ďalšími zariadeniami,
- zabezpečuje diagnostiku a bezpečné správanie.



## Robotika

Riadenie pohonov,  
spracovanie snímačov,  
komunikácia a  
bezpečnostné funkcie.

## Automobily

Motor, brzdy, airbag,  
osvetlenie, komfortné  
systémy a asistenčné  
funkcie.

## Medicína

Diagnostické zariadenia,  
dávkovanie liekov,  
monitorovanie a protetika.

## Priemysel

Výrobné stroje, regulátory,  
meniče, snímače a  
distribúované riadenie.

## Budovy

Vykurovanie, klimatizácia,  
zabezpečenie, meranie  
spotreby a automatizácia.

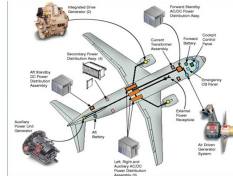
## Komunikácia

Prenos údajov, bezdrôtové  
siete, IoT zariadenia a  
vzdialená diagnostika.



Virtual Sound with surround.





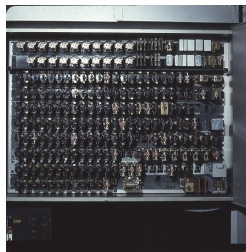
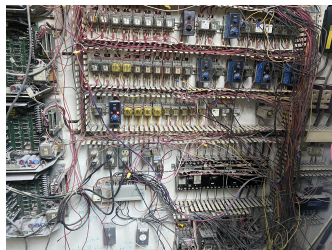
# 1. Historická línia

Od pevne zapojenej logiky k programovateľnému riadiacemu systému

## 💡 Podstata

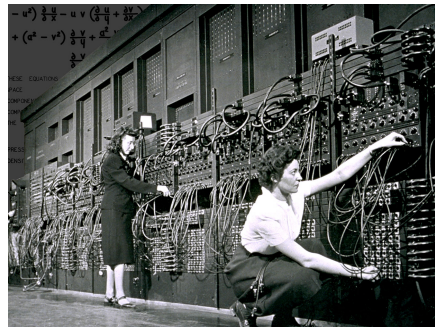
Pri pevne zapojenej logike je postup riadenia daný fyzickým prepojením kontaktov, relé, tlačidiel, časových relé a výkonových prvkov.

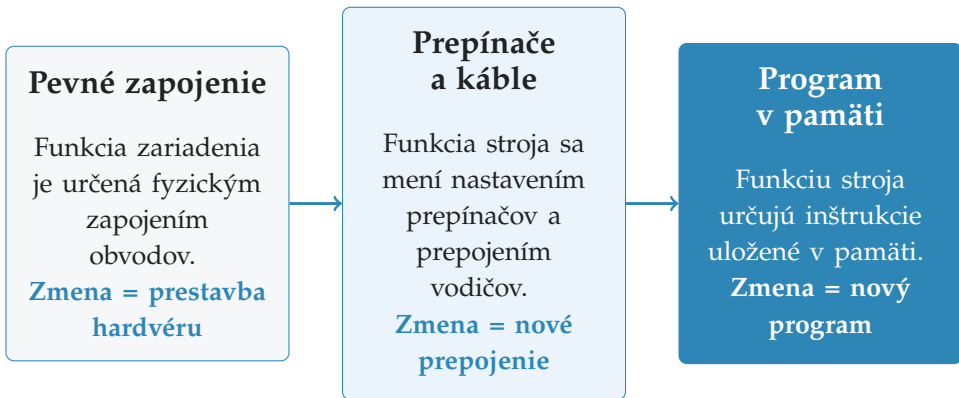
- riadiaca logika je zapísaná v elektrickej schéme,
- zmena algoritmu znamená zmenu zapojenia,
- vhodné pre jednoduché a stabilné úlohy,
- pri zložitejšom riadení rastie počet vodičov, relé a porúch,
- programovateľné riadiace systémy



- 12 / 48

- 1946: elektronický počítač založený na elektrónkach.
- Veľký výkon na svoju dobu, ale extrémne rozmery, spotreba a zložité programovanie.
- Programovanie často znamenalo prepájanie káblov a nastavovanie prepínačov.
- V r. 1948 upravený na čítanie z ROM: spomalenie výkonu 6-násobne, zmena dĺžky programovania z niekoľkých dní na hodiny
- Obsahoval: 21 501 elektróniek, 7200 kryštálových diód, 1500 relé, 70 000 rezistorov, 5 mil. ručne spájkovaných spojov, váha 30 t, rozmer 63 m<sup>2</sup>





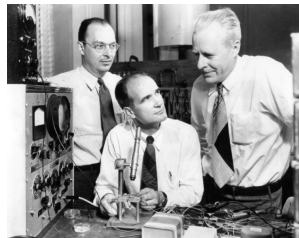
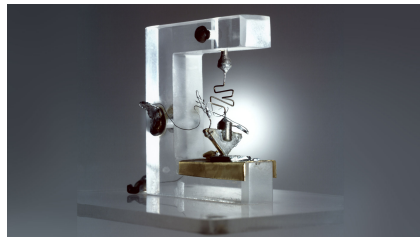
## 💡 Kľúčový posun

Zavedením programu uloženého v pamäti sa oddelila funkcia stroja od jeho fyzického zapojenia. Správanie systému bolo možné meniť úpravou programu bez prestavby hardvéru.

- 1947: tranzistor ako polovodičový spínací a zosilňovací prvok.
- Oproti elektrónkam priniesol menšie rozmery, nižšiu spotrebu a vyššiu spoľahlivosť.
- Umožnil vznik integrovaných obvodov a neskôr mikroprocesorov.

## Základná myšlienka

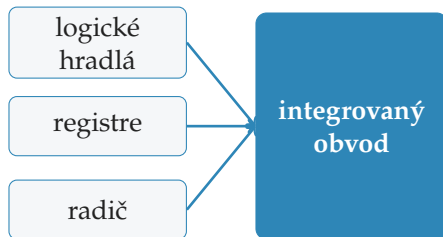
Tranzistory vytvárajú logické hradlá, registre, pamäte a riadiace obvody. Sú preto základnými stavebnými prvkami každého mikrokontroléra.



- Integrácia znamená, že tranzistory, vodiče a logické funkcie sú vytvorené na jednom čipe.
- Menej externých spojov prináša menšie rozmery, nižšiu cenu a vyššiu spoľahlivosť.
- Základná otázka sa zmenila: koľko funkcií dokážeme integrovať do jedného obvodu?

## Základná myšlienka

Integrovaný obvod presunul číslicovú logiku z veľkého množstva samostatných súčiastok do jedného čipu.



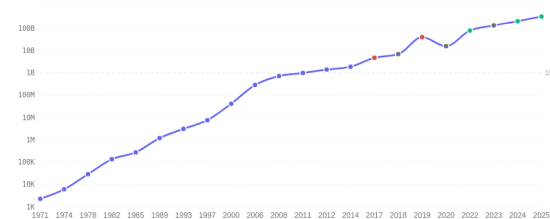
- Rast počtu tranzistorov umožnil vyšší výkon a nové funkcie.
- V mikroprocesoroch viedol k zložitejším CPU a väčším pamätiam.
- V embedded systémoch však nie je cieľom iba výkon: dôležitá je cena, spotreba, spoľahlivosť a periférie.

Transistor Count, 1971–2025 (Log Scale)

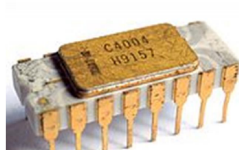
A straight line on a log scale means perfect exponential growth — Moore's Law held almost exactly for 40 years. Watch the line steepen after 2018 as Nvidia's AI chips push count far above the historical trend. Dot color shows which company set the record at each point.

Source: Wikipedia Transistor Count - Intel, AMD, Apple, Nvidia chip spec sheets

● Intel ● AMD ● Apple ● Nvidia



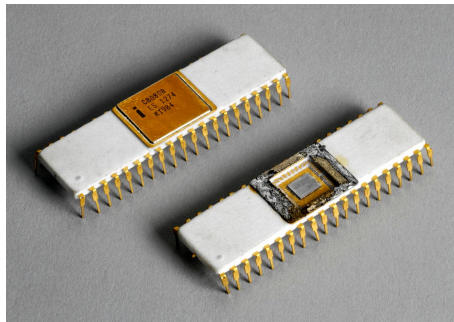
- 1971: komerčný 4-bitový mikroprocesor.
- Približne 2300 tranzistorov, 16-pinové púzdro.
- Vznikol v kontexte kalkulačiek firmy Busicom.
- Taktovacia frekvencia  $f = 740 \text{ kHz}$
- Dôležitý nie je iba výkon, ale nový princíp: procesor ako samostatný integrovaný obvod.

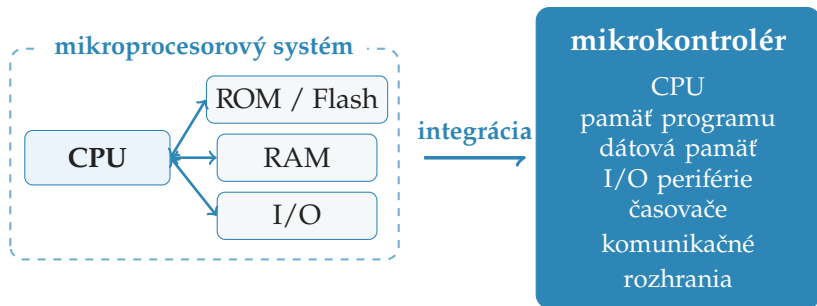


- 1972: 8-bitový mikroprocesor.
- Taktovacia frekvencia  $f = 500 - 800 \text{ kHz}$
- Prechod na 8 bitov umožnil prirodzene pracovať s jedným bajtom, napríklad so znakmi, malými celými číslami a stavovými údajmi.
- Vyžadoval však externé obvody: pamäte, vstupno-výstupné rozhrania a podpornú logiku.
- Ešte stále nejde o mikrokontrolér, ale o CPU, okolo ktorého bolo potrebné postaviť systém.



- 1974: výrazne výkonnejší 8-bitový mikroprocesor.
- Taktovacia frekvencia  $f = 2 \text{ MHz}$
- Používal sa v skorých mikropočítačoch a riadiacich aplikáciách.
- Ukázal, že mikroprocesor môže byť centrom všeobecného programovateľného systému.
- Pre embedded aplikácie však stále ostával problém veľkosti a počtu externých obvodov.





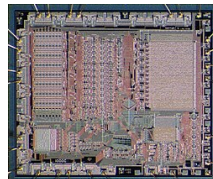
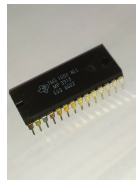
## Základná myšlienka

Mikrokontrolér vzniká integráciou procesora, pamätí a vstupno-výstupných periférií do jedného čipu.

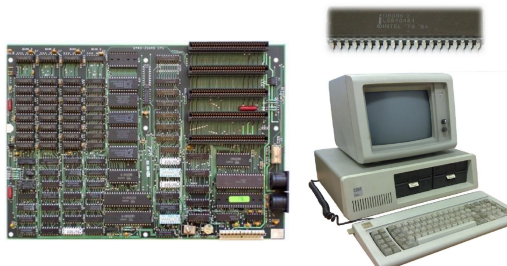
- 1974: rodina mikrokontrolérov Texas Instruments TMS1000.
- 4-bitové jadro, ROM, RAM a vstupno-výstupné linky v jednom integrovanom obvode.
- Vhodný pre lacné riadenie kalkulačiek, spotrebnej techniky a jednoduchých zariadení.
- Zlomová myšlienka: celý riadiaci systém môže byť integrovaný na jednom čipe.

## Základná myšlienka

TMS1000 spojil procesor, pamäte a vstupno-výstupné obvody do jedného cenovo



- 1978: 16-bitový mikroprocesor.
- Taktovacia frekvencia  $f = 5, 8, 10$  MHz
- Dôležitý pre osobné počítače a neskorší vývoj architektúry x86.
- Ukazuje inú vetvu vývoja: všeobecné počítače smerovali k rastu výkonu, pamäte a operačným systémom.
- Mikrokontroléry smerovali k integrácii, nízkej cene a priamej práci s okolím.



|                |                                                                                                                    |
|----------------|--------------------------------------------------------------------------------------------------------------------|
| 1978           | Intel 8086 – začiatok procesorovej línie x86.                                                                      |
| 1980           | Intel 8051 – významná rodina 8-bitových mikrokontrolérov pre vnorené systémy.                                      |
| 1983           | TMS32010 – prvý NMOS digitálny signálový procesor rodiny Texas Instruments TMS320.                                 |
| 1985           | ARM1 – jeden z prvých procesorov novej generácie architektúr RISC.                                                 |
| 1996           | Rodina AVR – 8-bitové mikrokontroléry s programovou pamäťou Flash.                                                 |
| 2005–2010      | Cortex-M3 a rozvoj rodiny ARM Cortex-M určenej pre výkonné, cenovo dostupné a energeticky úsporné mikrokontroléry. |
| 2016           | ESP32 – bezdrôtový SoC integrujúci procesor, Wi-Fi, Bluetooth a periférie na jednom čipe.                          |
| 2020           | Cortex-M55 a AI/ML akcelerátor Ethos-U55 ( <i>microNPU</i> ) pre lokálne vykonávanie neurónových sietí.            |
| 2021           | RP2040 – dvojjadrový mikrokontrolér spoločnosti Raspberry Pi, použitý na vývojovej doske Raspberry Pi Pico.        |
| 2022–súčasnosť | Cortex-M85 a RP2350 – rast výkonu a bezpečnosti; RP2350 umožňuje zvoliť jadrá Arm Cortex-M33 alebo Hazard3 RISC-V. |

## Podľa organizácie pamäte

- **von Neumannova** – spoločná pamäť pre program aj dáta
- **Harvardská** – oddelená pamäť programu a dát
- **modifikovaná Harvardská** – kombinácia oboch

## Podľa inštrukčnej sady

- **CISC** – zložitejšie inštrukcie, napr. x86
- **RISC** – jednoduchšie inštrukcie, napr. ARM, AVR, RISC-V
- **VLIW** – viac operácií v jednej inštrukcii

## Podľa šírky slova

- 4-bitové
- 8-bitové
- 16-bitové
- 32-bitové
- 64-bitové

## Podľa použitia

- všeobecné mikroprocesory
- mikrokontroléry
- DSP
- SoC
- špecializované akcelerátory



- **CISC:** rozsiahly súbor inštrukcií, často zložitejšie inštrukcie.
- **RISC:** jednoduchšie inštrukcie, viac registrov, dôraz na rýchle vykonávanie.
- AVR je 8-bitová RISC architektúra s 32 všeobecnými pracovnými registrami.
- Program v jazyku C prekladač prevedie na konkrétne strojové inštrukcie architektúry AVR.

## Dôležité

C kód v embedded systéme nie je izolovaný od hardvéru. Každý prístup do registra môže zmeniť správanie periférie.



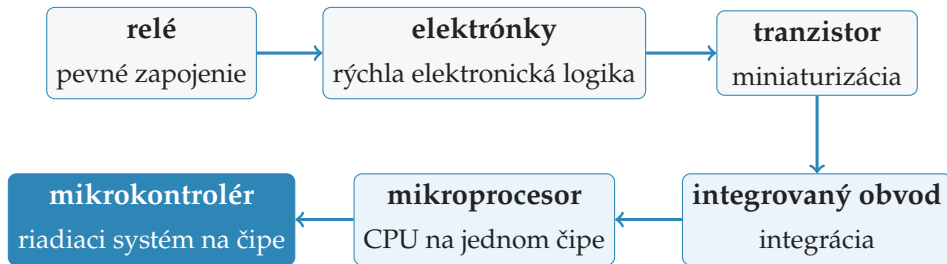
- ARM ukazuje smer k výkonným, ale energeticky efektívnym embedded systémom.
- V praxi sa používa od jednoduchých mikrokontrolérov až po aplikačné procesory v mobilných zariadeniach.
- Dnešný embedded svet zahŕňa 8-bitové, 16-bitové aj 32-bitové architektúry.

## Prečo AVR?

AVR je dostatočne jednoduchý na výučbu základov, ale pritom obsahuje reálne periférie používané v embedded praxi: GPIO, časovače, ADC, USART, SPI, TWI.



- 8-bitové AVR mikrokontroléry majú prehľadnú architektúru a dobrú dokumentáciu.
- Majú oddelenú pamäť programu a dát, 32 všeobecných registrov a hardvérové periférie.
- Ponúkajú dostatočne prehľadnú architektúru na výučbu základných princípov, pričom zároveň predstavujú plnohodnotné mikrokontroléry používané v reálnych embedded aplikáciách.
- ATmega328P je prakticky dostupný na doske Arduino UNO R3, čo znižuje problémy so zapojením.



## Základná myšlienka

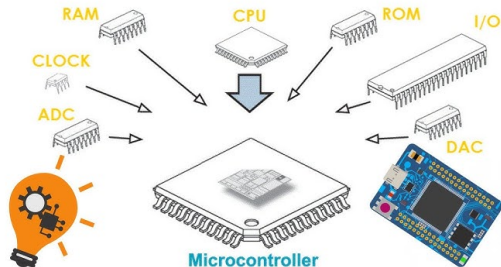
Mikrokontrolér vznikol postupnou miniaturizáciou a integráciou. Procesor, pamäte, vstupno-výstupné obvody a periférie sa podarilo spojiť do jedného cenovo dostupného a spoľahlivého čipu.

## 2. Základné pojmy

Pojmy, ktoré budeme používať pri práci s datasheetom a registrami.

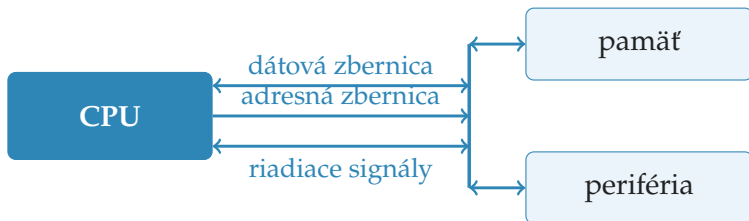
| Vlastnosť        | Mikroprocesor                            | Mikrokontrolér                  |
|------------------|------------------------------------------|---------------------------------|
| CPU              | samostatný procesorový čip               | CPU integrované v čipe          |
| Pamäť programu   | externá pamäť programu                   | pamäť programu v čipe           |
| Dátová pamäť     | externá RAM                              | RAM v čipe                      |
| Vstupy a výstupy | externé I/O obvody                       | GPIO a periférie v čipe         |
| Použitie         | všeobecný výpočtový systém               | riadenie konkrétneho zariadenia |
| Príklady         | Intel 8086, procesory osobných počítačov | ATmega328P, STM32, PIC, ESP32   |

- **CPU:** vykonáva inštrukcie.
- **Flash:** programová pamäť.
- **SRAM:** pracovná pamäť pre premenné programu.
- **EEPROM:** trvalo uložené konfiguračné údaje.
- **Periférie:** porty, časovače, ADC, komunikačné rozhrania.



| Pojem                   | Význam                                                                                                             |
|-------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Inštrukcia</b>       | Najmenší príkaz, ktorý vie procesor vykonať, napríklad presun údajov, sčítanie alebo skok v programe.              |
| <b>Inštrukčný súbor</b> | Množina všetkých inštrukcií, ktorým daný procesor rozumie a z ktorých sa skladá strojový program.                  |
| <b>ALU</b>              | Aritmeticko-logická jednotka vykonáva matematické a logické operácie, napríklad sčítanie, odčítanie, AND alebo OR. |
| <b>Register</b>         | Veľmi rýchle malé pamäťové miesto v procesore alebo periférii, s ktorým môže program priamo pracovať.              |

| Pojem                  | Význam                                                                                                              |
|------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Program Counter</b> | Programový čítač obsahuje adresu inštrukcie, ktorá sa má vykonať ako ďalšia.                                        |
| <b>Stack Pointer</b>   | Ukazovateľ zásobníka obsahuje adresu miesta v pamäti SRAM, kde sa práve nachádza vrchol zásobníka.                  |
| <b>SREG</b>            | Stavový register obsahuje príznaky výsledku operácie, napríklad nulový výsledok, prenos alebo povolenie prerušenia. |
| <b>Zásobník</b>        | Časť pamäte SRAM používaná na dočasné uloženie návratových adries, lokálnych údajov a stavu programu pri prerušení. |



- **Dátová zbernica** prenáša údaje medzi CPU, pamäťou a perifériami.
- **Adresná zbernica** vyberá pamäťovú bunku alebo register periférie, s ktorým sa bude pracovať.
- **Riadiace signály** určujú napríklad čítanie, zápis, prerušenie alebo stav pripojeného obvodu.

| Pojem                                                | Význam                                                                                                                                                                     |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hodinový signál</b><br><i>Clock signal</i>        | Pravidelný elektrický signál, ktorý synchronizuje činnosť procesora a určuje základné tempo jeho práce. Frekvencia sa udáva v hertzoch.                                    |
| <b>Hodinový cyklus</b><br><i>Clock cycle</i>         | Jedna úplná perióda hodinového signálu. Pri frekvencii 16 MHz trvá jeden hodinový cyklus 62,5 ns.                                                                          |
| <b>Inštrukcia</b><br><i>Instruction</i>              | Binárne zakódovaný príkaz procesora, napríklad presun údajov, aritmetická alebo logická operácia, prípadne skok v programe.                                                |
| <b>Inštrukčný cyklus</b><br><i>Instruction cycle</i> | Čas potrebný na načítanie a vykonanie jednej inštrukcie. Podľa typu inštrukcie môže trvať jeden alebo viac hodinových cyklov.                                              |
| <b>Strojový cyklus</b><br><i>Machine cycle</i>       | Architektúrne závislá základná časová jednotka činnosti procesora. Môže zahŕňať napríklad prístup do pamäte alebo inú čiastkovú operáciu potrebnú na vykonanie inštrukcie. |

| Pojem                                          | Význam                                                                                                                     |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Udalosť</b><br><i>Event</i>                 | Zmena, na ktorú má systém reagovať, napríklad stlačenie tlačidla, príchod impulzu zo snímača alebo dokončenie prevodu ADC. |
| <b>Latencia</b><br><i>Latency</i>              | Čas od vzniku udalosti po začiatok reakcie programu alebo hardvéru. Môže zahŕňať napríklad čakanie na obsluhu prerušenia.  |
| <b>Doba vykonania</b><br><i>Execution time</i> | Čas, ktorý procesor potrebuje na vykonanie určitej časti programu alebo konkrétnej úlohy.                                  |
| <b>Doba odozvy</b><br><i>Response time</i>     | Celkový čas od vzniku udalosti po dokončenie požadovanej reakcie. Zahŕňa latenciu aj dobu vykonania obsluhy.               |
| <b>Časový limit</b><br><i>Deadline</i>         | Najneskorší prípustný okamih, do ktorého musí byť reakcia alebo výpočet dokončený.                                         |
| <b>Časové kolísanie</b><br><i>Jitter</i>       | Kolísanie okamihu spustenia alebo periódy vykonávania úlohy. Je nežiaduce najmä pri vzorkovaní a späťoväzbovom riadení.    |

| Pojem                                                                       | Význam                                                                                                                               |
|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Systém reálneho času</b><br><i>Real-time system</i>                      | Systém, pri ktorom nestačí vypočítať správny výsledok. Výsledok musí byť dostupný aj v požadovanom čase.                             |
| <b>Tvrdý real-time</b><br><i>Hard real-time</i>                             | Nedodržanie časového limitu sa považuje za zlyhanie systému a môže viesť k nebezpečnému stavu.                                       |
| <b>Mäkký real-time</b><br><i>Soft real-time</i>                             | Občasné prekročenie časového limitu znižuje kvalitu činnosti, ale spravidla nespôsobí kritické zlyhanie systému.                     |
| <b>Periódá vzorkovania <math>T_s</math></b><br><i>Sampling period</i>       | Čas medzi dvoma po sebe idúcimi meraniami alebo výpočtami. Napríklad $T_s = 1$ ms znamená 1000 vzoriek za sekundu.                   |
| <b>Vzorkovacia frekvencia <math>f_s</math></b><br><i>Sampling frequency</i> | Počet vzoriek získaných za jednu sekundu. Platí $f_s = \frac{1}{T_s}$ .                                                              |
| <b>Časová rezerva</b><br><i>Timing margin</i>                               | Rozdiel medzi dostupným časom a skutočnou dobou vykonania úlohy. Rezerva umožňuje zvládnuť malé oneskorenia alebo kolísanie výpočtu. |

### 3. Programovacia platforma

Arduino UNO R3 ako doska  
ATmega328P ako mikrokontrolér  
AVR C ako spôsob programovania.



- UNO R3 rieši praktické veci: napájanie, USB, reset, hodinový zdroj, bootloader a konektory.
- Tým sa zníži počet chýb spôsobených breadboardom a vodičmi.
- Študent sa môže sústrediť na mikrokontrolér, registre a periférie.
- Doska je iba nosič ATmega328P, bez vyuzívania pripravených knižníc `digitalWrite()`.

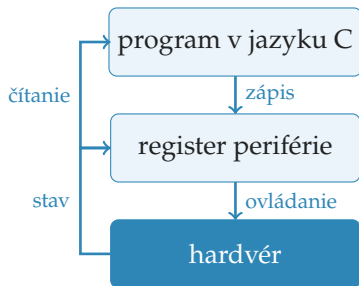
## Pravidlo predmetu

Hardvér Arduino UNO R3 áno. Arduino framework nie.

| Parameter         | Význam pre predmet                  |
|-------------------|-------------------------------------|
| 8-bit AVR RISC    | jednoduchá a čitateľná architektúra |
| 32 kB Flash       | program uložený v mikrokontroléri   |
| 2 kB SRAM         | premenné, zásobník, buffre          |
| 1 kB EEPROM       | trvalé konfiguračné dáta            |
| 23 GPIO liniek    | digitálne vstupy a výstupy          |
| 3 časovače/čítače | časovanie, PWM, meranie impulzov    |
| 10-bit ADC        | meranie analógových snímačov        |
| USART, SPI, TWI   | sériová komunikácia s okolím        |
| Watchdog          | zotavenie z poruchového stavu       |

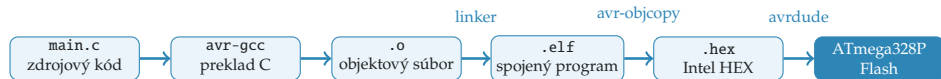
- 
- The diagram illustrates the internal architecture of an AVR CPU. At the center is the **AVR CPU**, which is connected to **SRAM** and an **OCD** (On-Chip Debugger). The CPU is also connected to a **debugWire** interface. The **Clock generation** block includes a 32.768kHz XOSC, a 8MHz Calib. RC, a 1MHz LP XOSC, and a 128kHz Int. OSC. The **Power Supervision POR/BOD & RESET** block is connected to VCC, RESET, and GND. The **ADC** (Analog-to-Digital Converter) is connected to AD06, AD07, PC05, AREF, AD0F, and AREF. The **EXTINT** (External Interrupt and Change Detection Controller) is connected to PORT0, PORT1, PORT2, PORT3, PORT4, PORT5, PORT6, and PORT7. The **TC0** (Timer Counter 0) is connected to TC0, TC0A, TC0B, TC0C, TC0D, TC0E, TC0F, TC0G, TC0H, TC0I, TC0J, TC0K, TC0L, TC0M, TC0N, TC0O, TC0P, TC0Q, TC0R, TC0S, TC0T, TC0U, TC0V, TC0W, TC0X, TC0Y, TC0Z, TC0AA, TC0AB, TC0AC, TC0AD, TC0AE, TC0AF, TC0AG, TC0AH, TC0AI, TC0AJ, TC0AK, TC0AL, TC0AM, TC0AN, TC0AO, TC0AP, TC0AQ, TC0AR, TC0AS, TC0AT, TC0AU, TC0AV, TC0AW, TC0AX, TC0AY, TC0AZ, TC0BA, TC0BB, TC0BC, TC0BD, TC0BE, TC0BF, TC0BG, TC0BH, TC0BI, TC0BJ, TC0BK, TC0BL, TC0BM, TC0BN, TC0BO, TC0BP, TC0BQ, TC0BR, TC0BS, TC0BT, TC0BU, TC0BV, TC0BW, TC0BX, TC0BY, TC0BZ, TC0CA, TC0CB, TC0CC, TC0CD, TC0CE, TC0CF, TC0CG, TC0CH, TC0CI, TC0CJ, TC0CK, TC0CL, TC0CM, TC0CN, TC0CO, TC0CP, TC0CQ, TC0CR, TC0CS, TC0CT, TC0CU, TC0CV, TC0CW, TC0CX, TC0CY, TC0CZ, TC0DA, TC0DB, TC0DC, TC0DD, TC0DE, TC0DF, TC0DG, TC0DH, TC0DI, TC0DJ, TC0DK, TC0DL, TC0DM, TC0DN, TC0DO, TC0DP, TC0DQ, TC0DR, TC0DS, TC0DT, TC0DU, TC0DV, TC0DW, TC0DX, TC0DY, TC0DZ, TC0EA, TC0EB, TC0EC, TC0ED, TC0EE, TC0EF, TC0EG, TC0EH, TC0EI, TC0EJ, TC0EK, TC0EL, TC0EM, TC0EN, TC0EO, TC0EP, TC0EQ, TC0ER, TC0ES, TC0ET, TC0EU, TC0EV, TC0EW, TC0EX, TC0EY, TC0EZ, TC0FA, TC0FB, TC0FC, TC0FD, TC0FE, TC0FF, TC0FG, TC0FH, TC0FI, TC0FJ, TC0FK, TC0FL, TC0FM, TC0FN, TC0FO, TC0FP, TC0FQ, TC0FR, TC0FS, TC0FT, TC0FU, TC0FV, TC0FW, TC0FX, TC0FY, TC0FZ, TC0GA, TC0GB, TC0GC, TC0GD, TC0GE, TC0GF, TC0GG, TC0GH, TC0GI, TC0GJ, TC0GK, TC0GL, TC0GM, TC0GN, TC0GO, TC0GP, TC0GQ, TC0GR, TC0GS, TC0GT, TC0GU, TC0GV, TC0GW, TC0GX, TC0GY, TC0GZ, TC0HA, TC0HB, TC0HC, TC0HD, TC0HE, TC0HF, TC0HG, TC0HH, TC0HI, TC0HJ, TC0HK, TC0HL, TC0HM, TC0HN, TC0HO, TC0HP, TC0HQ, TC0HR, TC0HS, TC0HT, TC0HU, TC0HV, TC0HW, TC0HX, TC0HY, TC0HZ, TC0IA, TC0IB, TC0IC, TC0ID, TC0IE, TC0IF, TC0IG, TC0IH, TC0II, TC0IJ, TC0IK, TC0IL, TC0IM, TC0IN, TC0IO, TC0IP, TC0IQ, TC0IR, TC0IS, TC0IT, TC0IU, TC0IV, TC0IW, TC0IX, TC0IY, TC0IZ, TC0JA, TC0JB, TC0JC, TC0JD, TC0JE, TC0JF, TC0JG, TC0JH, TC0JI, TC0JJ, TC0JK, TC0JL, TC0JM, TC0JN, TC0JO, TC0JP, TC0JQ, TC0JR, TC0JS, TC0JT, TC0JU, TC0JV, TC0JW, TC0JX, TC0JY, TC0JZ, TC0KA, TC0KB, TC0KC, TC0KD, TC0KE, TC0KF, TC0KG, TC0KH, TC0KI, TC0KJ, TC0KK, TC0KL, TC0KM, TC0KN, TC0KO, TC0KP, TC0KQ, TC0KR, TC0KS, TC0KT, TC0KU, TC0KV, TC0KW, TC0KX, TC0KY, TC0KZ, TC0LA, TC0LB, TC0LC, TC0LD, TC0LE, TC0LF, TC0LG, TC0LH, TC0LI, TC0LJ, TC0LK, TC0LL, TC0LM, TC0LN, TC0LO, TC0LP, TC0LQ, TC0LR, TC0LS, TC0LT, TC0LU, TC0LV, TC0LW, TC0LX, TC0LY, TC0LZ, TC0MA, TC0MB, TC0MC, TC0MD, TC0ME, TC0MF, TC0MG, TC0MH, TC0MI, TC0MJ, TC0MK, TC0ML, TC0MM, TC0MN, TC0MO, TC0MP, TC0MQ, TC0MR, TC0MS, TC0MT, TC0MU, TC0MV, TC0MW, TC0MX, TC0MY, TC0MZ, TC0NA, TC0NB, TC0NC, TC0ND, TC0NE, TC0NF, TC0NG, TC0NH, TC0NI, TC0NJ, TC0NK, TC0NL, TC0NM, TC0NN, TC0NO, TC0NP, TC0NQ, TC0NR, TC0NS, TC0NT, TC0NU, TC0NV, TC0NW, TC0NX, TC0NY, TC0NZ, TC0OA, TC0OB, TC0OC, TC0OD, TC0OE, TC0OF, TC0OG, TC0OH, TC0OI, TC0OJ, TC0OK, TC0OL, TC0OM, TC0ON, TC0OO, TC0OP, TC0OQ, TC0OR, TC0OS, TC0OT, TC0OU, TC0OV, TC0OW, TC0OX, TC0OY, TC0OZ, TC0PA, TC0PB, TC0PC, TC0PD, TC0PE, TC0PF, TC0PG, TC0PH, TC0PI, TC0PJ, TC0PK, TC0PL, TC0PM, TC0PN, TC0PO, TC0PP, TC0PQ, TC0PR, TC0PS, TC0PT, TC0PU, TC0PV, TC0PW, TC0PX, TC0PY, TC0PZ, TC0QA, TC0QB, TC0QC, TC0QD, TC0QE, TC0QF, TC0QG, TC0QH, TC0QI, TC0QJ, TC0QK, TC0QL, TC0QM, TC0QN, TC0QO, TC0QP, TC0QQ, TC0QR, TC0QS, TC0QT, TC0QU, TC0QV, TC0QW, TC0QX, TC0QY, TC0QZ, TC0RA, TC0RB, TC0RC, TC0RD, TC0RE, TC0RF, TC0RG, TC0RH, TC0RI, TC0RJ, TC0RK, TC0RL, TC0RM, TC0RN, TC0RO, TC0RP, TC0RQ, TC0RR, TC0RS, TC0RT, TC0RU, TC0RV, TC0RW, TC0RX, TC0RY, TC0RZ, TC0SA, TC0SB, TC0SC, TC0SD, TC0SE, TC0SF, TC0SG, TC0SH, TC0SI, TC0SJ, TC0SK, TC0SL, TC0SM, TC0SN, TC0SO, TC0SP, TC0SQ, TC0SR, TC0SS, TC0ST, TC0SU, TC0SV, TC0SW, TC0SX, TC0SY, TC0SZ, TC0TA, TC0TB, TC0TC, TC0TD, TC0TE, TC0TF, TC0TG, TC0TH, TC0TI, TC0TJ, TC0TK, TC0TL, TC0TM, TC0TN, TC0TO, TC0TP, TC0TQ, TC0TR, TC0TS, TC0TT, TC0TU, TC0TV, TC0TW, TC0TX, TC0TY, TC0TZ, TC0UA, TC0UB, TC0UC, TC0UD, TC0UE, TC0UF, TC0UG, TC0UH, TC0UI, TC0UJ, TC0UK, TC0UL, TC0UM, TC0UN, TC0UO, TC0UP, TC0UQ, TC0UR, TC0US, TC0UT, TC0UU, TC0UV, TC0UW, TC0UX, TC0UY, TC0UZ, TC0VA, TC0VB, TC0VC, TC0VD, TC0VE, TC0VF, TC0VG, TC0VH, TC0VI, TC0VJ, TC0VK, TC0VL, TC0VM, TC0VN, TC0VO, TC0VP, TC0VQ, TC0VR, TC0VS, TC0VT, TC0VU, TC0VV, TC0VW, TC0VX, TC0VY, TC0VZ, TC0WA, TC0WB, TC0WC, TC0WD, TC0WE, TC0WF, TC0WG, TC0WH, TC0WI, TC0WJ, TC0WK, TC0WL, TC0WM, TC0WN, TC0WO, TC0WP, TC0WQ, TC0WR, TC0WS, TC0WT, TC0WU, TC0WV, TC0WW, TC0WX, TC0WY, TC0WZ, TC0XA, TC0XB, TC0XC, TC0XD, TC0XE, TC0XF, TC0XG, TC0XH, TC0XI, TC0XJ, TC0XK, TC0XL, TC0XM, TC0XN, TC0XO, TC0XP, TC0XQ, TC0XR, TC0XS, TC0XT, TC0XU, TC0XV, TC0XW, TC0XX, TC0XY, TC0XZ, TC0YA, TC0YB, TC0YC, TC0YD, TC0YE, TC0YF, TC0YG, TC0YH, TC0YI, TC0YJ, TC0YK, TC0YL, TC0YM, TC0YN, TC0YO, TC0YP, TC0YQ, TC0YR, TC0YS, TC0YT, TC0YU, TC0YV, TC0YW, TC0YX, TC0YY, TC0YZ, TC0ZA, TC0ZB, TC0ZC, TC0ZD, TC0ZE, TC0ZF, TC0ZG, TC0ZH, TC0ZI, TC0ZJ, TC0ZK, TC0ZL, TC0ZM, TC0ZN, TC0ZO, TC0ZP, TC0ZQ, TC0ZR, TC0ZS, TC0ZT, TC0ZU, TC0ZV, TC0ZW, TC0ZX, TC0ZY, TC0ZZ.

- Register je malé pamäťové miesto s pevne danou adresou.
- Program môže do registra zapisovať alebo z neho čítať.
- Zápisom do registra sa nastavuje alebo ovláda hardvér.
- Čítaním registra program zisťuje aktuálny stav hardvéru.



## Základná myšlienka

Register je rozhranie, prostredníctvom ktorého program komunikuje s perifériami mikrokontroléra.



| Prvok       | Význam                                                                               |
|-------------|--------------------------------------------------------------------------------------|
| main.c      | Zdrojový program v jazyku C.                                                         |
| avr-gcc     | Preloží C kód pre architektúru AVR a vytvorí objektový súbor .o.                     |
| .o          | Objektový súbor: preložená časť programu, ešte nie celý výsledný program.            |
| linker      | Spojí objektové súbory, knižnice a štartovací kód do súboru .elf.                    |
| .elf        | Výsledný spojený program vrátane strojového kódu a pomocných informácií pre ladenie. |
| avr-objcopy | Zo súboru .elf vytvorí súbor .hex určený na nahratie.                                |
| avrdude     | Softvér, ktorý preniesie .hex do Flash pamäte mikrokontroléra.                       |

- Jazyk C je v embedded systémoch dôležitý, pretože umožňuje pracovať s hardvérom a zároveň písať čitateľný program.
- V tomto predmete sa jazyk C nebude používať iba ako všeobecný programovací jazyk, ale ako nástroj na riadenie periférií mikrokontroléra.
- Študent musí rozumieť premenným, dátovým typom, bitovým operáciám, funkciám, ukazovateľom, hlavičkovým súborom a práci s registrami.

## Základná myšlienka

Program v jazyku C je preložený na strojový kód. Mikrokontrolér nevykonáva priamo jazyk C, ale binárne inštrukcie uložené vo Flash pamäti.



- inštalácia a kontrola VS Code, PlatformIO, AVR toolchainu,
- vytvorenie projektu pre dosku uno,
- kompilácia minimálneho programu,
- nahratie pripraveného programu do UNO R3,
- orientácia na doske: napájanie, USB, reset, digitálne a analógové piny,
- vysvetlenie mapovania Arduino pinov na porty ATmega328P.



- Vývoj smeroval od pevne zapojenej logiky k programovateľnému riadeniu.
- Mikroprocesor integroval CPU na jeden čip, mikrokontrolér integroval celý riadiaci systém.
- Embedded systém spája program, elektroniku, snímače, aktuátory a časovanie.
- Arduino UNO R3 používame ako hardvérovú dosku s ATmega328P.
- Programovať budeme AVR C, teda priamo cez registre a datasheet.

Ďakujem za  
pozornosť